

Unsupervised learning for text mining: topic modelling and vector embeddings

Fedor Duzhin

class 3



- 1 Intro
- 2 Topic modelling
- 3 Word embeddings
- 4 GloVe
- 5 Summary

Section 1

Intro

Overview

We are now familiar with algorithms for supervised machine learning — linear regression (for regression) and logistic regression (for classification). We know that it is important to split the data into a training set and a test set. To prevent overfitting, it may be important to use regularization for linear or logistic regression if the number of features is very large.

These models are trained by minimizing a suitable loss function that, roughly speaking, shows how the model's predictions deviate from the observed data.

Today we will continue with unsupervised learning.

Learning outcomes of today's class

- 1 Explain Latent Dirichlet Allocation for topic modelling.
- 2 Explain the concept of word embedding — an unsupervised approach to natural language processing.
- 3 Explain how to compute the term co-occurrence matrix, assign word to vectors in a way consistent with the co-occurrence matrix, find the loss function, and train GloVe.

Section 2

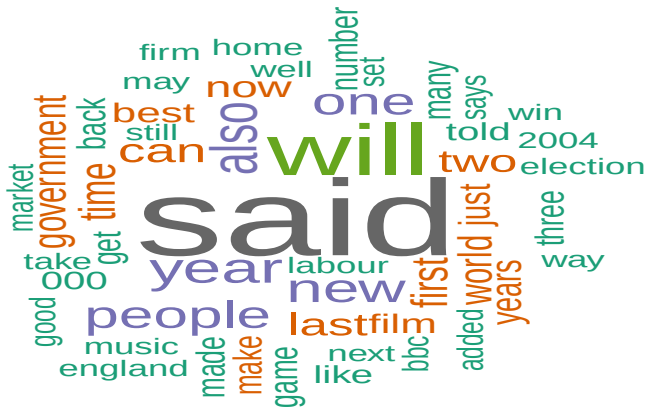
Topic modelling

Problem statement

We have a collection of texts and we want to get some understanding of key themes that are covered in these texts.

This is known as *topic modelling* and one of most popular algorithms is *Latent Dirichlet Allocation*.

As an example, consider a collection of BBC news:



Here, we know the right answer — each news article has a pre-assigned category. But let us pretend that we don't know it.

Table 1: Frequency of news categories

Category	Freq
business	336
entertainment	273
politics	274
sport	346
tech	261

Main principles

The main ideas are

- Every document is a mixture of topics
- Every topic is a mixture of words

LDA output

We will train an LDA model with 5 topics (the number of topics is a hyperparameter, but in this example we know the right answer).

Table 2: Topic probabilities

document	1	2	3	4	5
1	0.0	0	1	0.0	0.0
2	0.0	0	0	0.0	1.0
3	0.3	0	0	0.0	0.7
4	0.0	1	0	0.0	0.0
5	0.0	0	1	0.0	0.0
6	0.3	0	0	0.7	0.0

The sum of entries of every row is 1

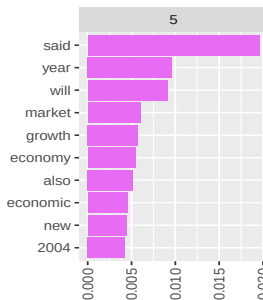
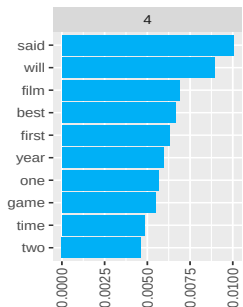
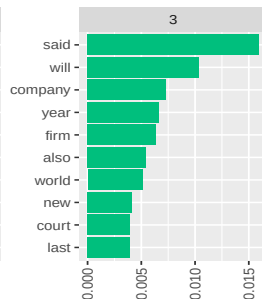
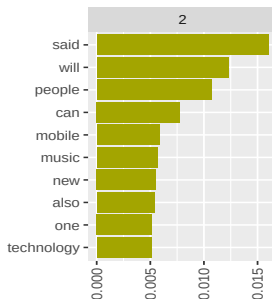
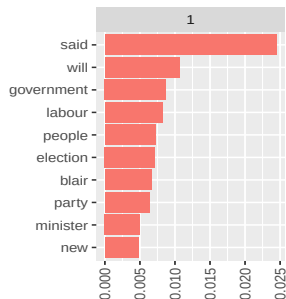
And here is a sample of term probabilities

Table 3: Term probabilities

term	1	2	3	4	5
accounting	0e+00	0e+00	0.00122	0e+00	0.00000
chemicals	0e+00	3e-05	0.00000	0e+00	0.00012
deserves	2e-05	0e+00	0.00000	7e-05	0.00000
destroyed	4e-05	1e-05	0.00010	4e-05	0.00005
savage	0e+00	0e+00	0.00004	6e-05	0.00000

The sum of entries of every column is 1

10 most frequent words in each topic:



Strictly speaking, we shouldn't do it (why?). But if we assign the topic with the highest probability to each document, then we will get the following:

business	entertainment	politics	sport	tech
7	7	253	3	11
8	14	1	0	227
86	32	3	47	14
0	215	2	295	7
235	5	15	1	2

How the model works

Input:

- M — the number of documents
- N — the number of words in a document
- k — the number of topics
- V — size of the vocabulary

Parameters of the model:

- D , an $M \times K$ matrix of topic distributions in a document
- W , a $V \times K$ matrix of word distributions in a topic

Simple worked example: 9 words, 3 topics — A , B , C , 2 documents with 5 and 4 words and the following mixture of topics:

Table 4: Topic probabilities in a document (matrix D)

document	No of words	Topic A prob	Topic B prob	Topic C prob
Doc 1	5	0.6	0.3	0.1
Doc 2	4	0.1	0.2	0.7

First, we generate placeholders for topics in our documents:

		1	2	3	4	5
.	Doc 1	A	B	B	A	A

		1	2	3	4
.	Doc 2	C	A	C	B

And we have the following distribution of word probabilities for each of the three topics:

Table 5: Word probabilities for topics (matrix W)

word	A word prob	B word prob	C word prob
covid	0.250	0.025	0.025
coronavirus	0.250	0.050	0.050
epidemics	0.250	0.075	0.025
education	0.050	0.250	0.025
university	0.050	0.250	0.025
students	0.025	0.225	0.025
finance	0.050	0.025	0.250
banking	0.050	0.050	0.275
recession	0.025	0.050	0.300

And then we randomly replace placeholders with words

Table 6: Doc 1

topic	A	B	B	A	A
word	coronavirus	university	coronavirus	epidemics	covid

Table 7: Doc 2

topic	C	A	C	B
word	finance	coronavirus	banking	students

The order of words in a document is ignored — *bag of words*.

Training LDA

In practice, we have a collection of documents and a vocabulary, i.e., we know the document-term matrix. We are assigning a value to k , the number of topics.

Topics are latent, i.e., invisible.

Probabilities of topics in a document and words in a topic are assumed to follow the Dirichlet distribution.

And there is a loss function — Kullback–Leibler divergence of the theoretical distribution and observed data.

Details are beyond our course.

Revision of Notebooks 3 and 4 — WooClap

*<https://app.wooclap.com/LKQBAS>



Figure 1: Scan this QR code to open WooClap activity

Section 3

Word embeddings

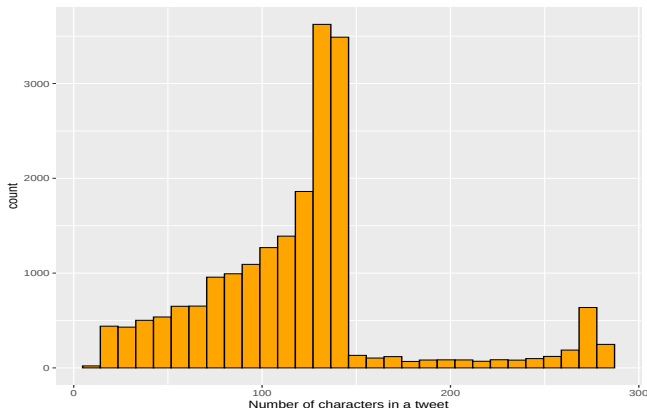
Motivation

Here is a sample of Trump's twitter.

text	created_at	retweet_count
Join me in North Carolina - tomorrow at 7:30pm! #ImWithYou Tickets: https://t.co/vDag45zrDc	08-17-2016 13:24:41	3585
@RogerBiles @EricLahti929	04-10-2013 18:26:27	0
@nbc Thanks! @Chiamboy8 Thanks for your kind thoughts I love S.I.	02-01-2013 20:22:07	1
@ET614: @realDonaldTrump Trump For President 2016 !!!	09-08-2013 03:06:09	29
Thanks! @LeighAnnMac Send his info to my office.	09-19-2013 02:34:07	0

The number of tweets is 20112

Distribution of the number of characters in a tweet:

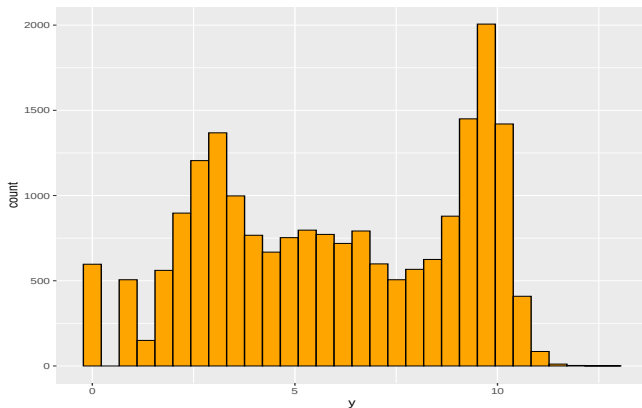


We will now clean the text by changing everything to lower case and deleting all characters except for alphabet and numbers.

Here is the word cloud:



Histogram of $\log(\text{retweet_count} + 1)$:



Document-term matrix is the matrix M whose rows represent documents (tweets) and columns words in our vocabulary. Then M_{ij} is the number of times the word j appears in document i .

Here is a sample of DTM for Trump's twitter:

for	day	first	see	thank	how	stock
2	0	1	0	0	0	0
0	0	0	0	1	0	0
0	0	0	0	0	0	0
0	0	0	0	1	0	0
0	1	0	0	0	0	0
0	0	0	0	0	1	1

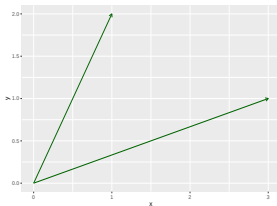
Dimensions of DTM are 20112×28337 .

The number of features is very large. Even if we only include terms that appear in, say, at least 10 tweets, the number of features is still 3067.

Overview of math basics

If we have two vectors $x, y \in \mathbb{R}^n$, their dot product is

$$x \cdot y = \sum_{i=1}^n x_i y_i$$



At the same time, we have

$$x \cdot y = \|x\| \times \|y\| \times \cos \alpha$$

And we also have $\|x\|^2 = x \cdot x$ and $\|y\|^2 = y \cdot y$.

The cosine similarity is the cosine of the angle between two vectors. It is calculated by

$$\cos \alpha = \frac{x \cdot y}{\sqrt{x \cdot x} \times \sqrt{y \cdot y}}$$

Vectors pointing in the same direction have cosine similarity 1. Vectors pointing in the opposite direction have cosine similarity -1 . Orthogonal vectors have cosine similarity 0.

A *group* G is a set together with an operation $*$ satisfying three conditions:

- ① The operation is associative, i.e., $(a * b) * c = a * (b * c)$.
- ② There exists the *identity* element e such that $x * e = e * x = x$ for all $x \in G$.
- ③ For each $x \in G$, there exists the *inverse* element x^{-1} such that $x * x^{-1} = x^{-1} * x = e$.

A *homomorphism* $\varphi : G_1 \rightarrow G_2$ is a function that preserves the group operation, i.e.,

$$\varphi(x_1 * x_2) = \varphi(x_1) * \varphi(x_2)$$

Word embeddings

In order to use machine learning algorithms to process text, we need to convert text to a vector of features.

The conceptual idea behind word embeddings is that we assign a vector to every word in our dataset (language data sets are called 'corpora', singular 'corpus') such that words that appear in similar contexts get vectors with high cosine similarity.

Moreover, vector space structure is consistent with semantics.

Ideally, we want to get something like

$$v(\text{France}) + v(\text{capital}) = v(\text{Paris})$$

Or

$$v(\text{man}) - v(\text{woman}) = v(\text{brother}) - v(\text{sister})$$

Section 4

GloVe

Word co-occurrence

Notations:

- X_{ij} — number of times word j occurs in the context of word i
- $X_i = \sum_j X_{ij}$ — number of times any word occurs in the context of word i .
- $P_{ij} = P(j|i) = \frac{X_{ij}}{X_i}$ — probability that word j occurs in the context of word i .

Example:

baa baa black sheep have you any wool yes sir yes sir three bags full

For demonstration, the window size is 2, i.e., one word occurs in the context of another word whenever they are next to each other or there is one word between them.

	any	bags	black	full	have	sheep	three	wool	you	baa	sir	yes
any	0	0	0	0	1	0	0	1	1	0	0	0
bags	0	0	0	1	0	0	1	0	0	0	1	0
black	0	0	0	0	1	1	0	0	0	2	0	0
full	0	1	0	0	0	0	1	0	0	0	0	0
have	1	0	1	0	0	1	0	0	1	0	0	0
sheep	0	0	1	0	1	0	0	0	1	1	0	0
three	0	1	0	1	0	0	0	0	0	0	1	1
wool	1	0	0	0	0	0	0	0	1	0	0	0
you	1	0	0	0	1	1	0	1	0	0	0	0
baa	0	0	2	0	0	1	0	0	0	1	0	0
sir	0	1	0	0	0	0	1	0	0	0	1	3
yes	0	0	0	0	0	0	1	0	0	0	3	1

The following real example is taken from the GloVe paper.

Expression	$P(k \text{ice})$	$P(k \text{steam})$	$P(k \text{ice}) / P(k \text{steam})$
k=solid	0.00019	0.00002	8.900
k=gas	0.00007	0.00078	0.085
k=water	0.00300	0.00220	1.360
k=fashion	0.00002	0.00002	0.960

It shows that it is important to consider how often two different words occur in the context of another word and then take ratios of probabilities.

GloVe

In GloVe model, we have word vectors w and context vectors $\tilde{w}$.

Let w_i and w_j be word vectors for words i and j and let $\tilde{w}_k$ be the context vector for word k . The general model is

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

To reflect the vector space structure on $\mathbb{R}^d$, we choose F so that it only depends on the difference of target vectors, i.e.,

$$F(w_i - w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

The function F has a vector input and a scalar output. To further simplify it, we will make the assumption that it actually depends on the dot product of the vectors, i.e.,

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

We are looking for a function $F : \mathbb{R} \rightarrow \mathbb{R}$ such that

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

This happens if F is a homomorphism from $(\mathbb{R}, +)$ to $(\mathbb{R}_{>0}, \times)$.

In that case, we will have

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{F(w_i^T \tilde{w}_k)}{F(w_j^T \tilde{w}_k)}$$

This will happen if

$$F(w_i^T \tilde{w}_k) = P_{ik} = \frac{X_{ik}}{X_i}$$

A familiar homomorphism from $(\mathbb{R}, +)$ to $(\mathbb{R}_{>0}, \times)$ is $\exp$.

Ideally, we have

$$w_i^T \tilde{w}_k = \log X_{ik} - \log X_i$$

Now let $b_i = \log X_i$ (bias).

Note that the roles of a word and a context word are symmetric. To restore the symmetry, we will add a bias for k and finally get

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log X_{ik}$$

And then we minimize the loss function:

$$L(w, \tilde{w}, b, \tilde{b}) = \sum_{i,k=1}^V f(X_{ik}) (w_i^T \tilde{w}_k + b_i + \tilde{b}_k - \log X_{ik})^2,$$

where V is the vocabulary size and $f(X_{ik})$ is a weighting function chosen carefully to avoid overweighting too rare and too common co-occurrences.

The final word embedding is simply

$$v_i = w_i + \tilde{w}_i$$

Here is a small example:

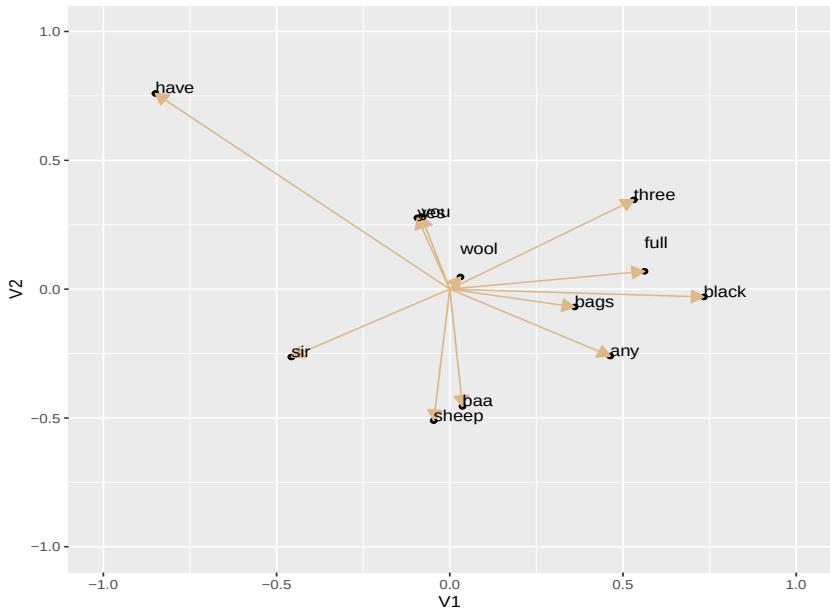
baa baa black sheep have you any wool yes sir yes sir three bags full

We train the model for 5 epochs with window size 2 and 2-dimensional vectors.

Here are v_i :

any	0.46	-0.26
bags	0.36	-0.07
black	0.73	-0.03
full	0.56	0.07
have	-0.85	0.76
sheep	-0.05	-0.51
three	0.53	0.35
wool	0.03	0.05
you	-0.08	0.28
baa	0.04	-0.46
sir	-0.46	-0.26
yes	-0.09	0.28

Cosine similarity between “yes” and “sir” is -0.196



GloVe combined with other algorithms

GloVe assigns vectors to words, i.e., defines a map

$$v : \text{vocabulary} \rightarrow \mathbb{R}^d$$

Usually we want to assign a vector to the whole document:

$$v(\text{document}) = \sum_{x \in \text{document}} v(x)$$

After vectors are assigned to entire documents, we can do downstream NLP tasks.

Activity: pre-trained GloVe

Notebook 5 — graded teamwork.

Planned time: 1.5 hours.

Section 5

Summary

Main takeaways

In unsupervised learning, there is no response variable. We can't estimate and compare accuracy of different unsupervised learning models (like we compare accuracy of, say, logistic regression and random forest). Different unsupervised learning models serve different purpose. Therefore, usually it doesn't make sense to split the data into training and test sets in unsupervised learning.

However, unsupervised models have parameters and hyper-parameters, just like supervised models. Parameter values are found by minimizing a suitable loss function. Hyperparameter values are set by trial-and-error.

LDA:

- Parameters — probability matrices D and W .
- Hyperparameter — the number of topics, k .

To train ML models that are capable of processing text, we need to convert text to a vector of features. A fruitful idea is to assign a vector to every word (word embedding) and assign the sum of all word vectors to a text.

Word embeddings are trained in an unsupervised manner by minimizing a suitable loss function. Then we can build other supervised models on top of a word embedding.

GloVe:

- Parameters — word vectors, context vectors, and bias scalars for all the words in the vocabulary.
- Hyperparameters — dimension of word vectors, window size, weights for TCM, weight coefficient for the loss function.

Concepts and ideas

- 1 Topic modelling
- 2 Topic as a mixture of words
- 3 Document as a mixture of topics
- 4 Word embedding
- 5 Co-occurrence matrix
- 6 Algorithm for unsupervised learning: GloVe (constructing word vectors)
- 7 Cosine similarity

Feedback form



Figure 2: Scan this QR code to do feedback on week 3

Please fill this feedback form:

- <https://forms.gle/uSqTowTm9DwETrb59>

At home

- Notebook 6